COMPSCI5011 — Information Retrieval · 2026 EXAM CHEATSHEET

University of Glasgow · All key formulas, traps & methods · Based on 2022–2025 papers

CLOSED BOOK EXAM

1. Inverted Index & TF-IDF

Structure: term → [(docID, tf), ...] + df per term

w(t,d) = tf x log2(N/df) [on cribsheet]

WHEN: Always — foundation of all retrieval ranking

! KEY TRAPS:

- If df = N → IDF = log(1) = 0 → term scores ZERO everywhere
- Check every query term df before computing scores!
- tf in Doc = count AFTER stopword removal (recount!)
- rank = sort docs by score descending; ties = both same rank

2. Vector Space Model — Cosine Similarity

cos(D,Q) = (D·Q) / (|D| x |Q|) [on cribsheet]

Steps:

- 1. Dot product: sum(d_i x q_i) — only matching terms contribute
- 2. |D| = sqrt(sum of ALL term weight squares in doc)
- 3. |Q| = sqrt(sum of query term weight squares)
- Shorter focused docs score higher than long docs with same dot product!

3. Evaluation Metrics

Precision = |Rel∩Ret| / |Ret| Recall = |Rel∩Ret| / |Rel|

F1 = 2PR/(P+R) P@k = relevant in top-k / k

AP = (1/|R_total|) x sum P(k) at EACH relevant rank

DCG = rel1 + sum(reli/log2(i+1)) nDCG = DCG/IDCG

! CRITICAL EXAM TRAPS:

- AP: ALWAYS divide by TOTAL relevant (not retrieved relevant)
- Non-retrieved relevant docs contribute 0 to AP — they still count!
- From R-P graph: rank_k = k / P(at that recall point)
- Interpolated P(r) = MAX precision at recall >= r (look ahead!)
- IDCG = ideal DCG (sort by relevance grade descending)

4. Heap's Law & Zipf's Law

Heap: V = K x sqrt(n) Zipf: r x P(w|C) = A

- Heap: n=tokens(docs x avgdl). Find K=V/sqrt(n). V_new=K*sqrt(n_new)
- Larger collections → fewer NEW terms per doc (sublinear growth)
- Zipf: A = rank x freq/total. Then P(w1)=A/1, P(w2)=A/2
- Top 4 words cover ~20% of all tokens — very skewed distribution

5. Rocchio Relevance Feedback

q_new = a*q0 + (b/|Dr|)*SUM(Dr) - (g/|Dn|)*SUM(Dn) [on cribsheet]

Typical: a=1 b=0.75 g=0.25 Set NEGATIVE results to 0

EXAM STEPS:

- 1. Write q_orig vector (raw TF of query terms)
- 2. Relevant centroid = average of relevant doc vectors
- 3. Non-relevant centroid = average of non-rel doc vectors
- 4. Apply formula term by term → drop negatives

"Find Similar" feature: a=0, b=1, g=0

→ Ignore original query, use ONLY clicked doc as new query

6. Language Models — JM Smoothing

P(w|d) = (1-L) x tf(w,d)/|d| + L x P(w|C) [on cribsheet]

Score(q|d) = PRODUCT of P(w|d) for all query terms

- WHY: Zero prob fix — tf=0 still gets L x P(w|C) > 0
- Rare terms (low P(w|C)) discriminate MORE — favour their tf
- Lambda=0.5 → equal doc vs collection weight

! Doc with higher tf of RARER term ranks above doc with higher tf of common

- Bayes: P(D|Q) rank= P(D) x P(Q|D) [uniform prior → P(D) drops out]

7. BIM / Probabilistic Retrieval (RSV)

RSV = SUM log[pi(1-si) / si(1-pi)] for terms where d_i = 1

No relevance info: pi = 0.5 si = df / N

With relevance (R docs known):

pi=(ri+0.5)/(R+1) si=(df-ri+0.5)/(N-R+1) [Laplace smoothing]

! EXAM TRAPS:

- Only sum for terms where d_i=1 (term PRESENT in doc)
- Relevant doc without query term → RSV = 0 (BIM limitation)
- Non-relevant doc WITH query term scores same as relevant doc

8. IR Assumptions (4 marks — always asked!)

- 1. Term independence: "New York" ≠ "New"+"York"
- 2. Document independence: ignores diversity/novelty
- 3. Static relevance: ignores time (2019 COVID doc misleading today)
- 4. Topical = User relevance: ignores readability/credibility

→ **For EACH: give formula/model + concrete real-world example!**

→ **1 mark per assumption — no example = half mark**

9. HITS Algorithm — Hub & Authority Scores

Hub = links TO many good authorities **Authority = linked BY many good hubs**

a(p) = SUM h(q) for all q->p h(p) = SUM a(q) for all p->q

MATRIX METHOD (exam: "using matrices — do NOT resolve"):

- 1. Build adjacency matrix A: A[i][j]=1 if page i links TO page j
- 2. Compute transpose A^T
- 3. M_authority = A^T x A → eigenvalue eqn: M_a * a = L * a
- 4. M_hub = A x A^T → eigenvalue eqn: M_h * h = L * h
- 5. Diagonal of M_a[i][i] = in-degree of page i (quick sanity check!)

ITERATIVE METHOD (2 iterations):

- Start h=(1,1,...). Update ALL a values, then ALL h values. Normalise.
- Normalise by MAX value (divide all scores by largest score)

10. TAAT vs DAAT Index Traversal

TAAT (Term-At-A-Time):

- Process one FULL posting list, then next. O(N) memory — bad!

DAAT (Document-At-A-Time):

- Process all terms for ONE doc, then next. O(K) memory — good!

MaxScore / WAND (dynamic pruning):

- Precompute UB(t)=max score contribution. Skip doc if partial+remaining_UBs <

11. Index Compression — Lossy vs Lossless

LOSSLESS — no info lost, retrieval unchanged:

- d-gaps (delta encode docIDs), VByte, Gamma codes, PForDelta
- Measure: compression ratio + decompression speed (no MAP change)

LOSSY — some info discarded:

- Static pruning (remove low-weight postings), stopwords, quantisation
- Measure: compression ratio AND MAP/nDCG before vs after

12. Exam Strategy — 1h 30min, 60 marks

Q1 ~25 min	Inverted index → TF-IDF → Cosine → P/R/AP/DCG
Q2 ~25 min	Heap's → Rocchio → JM Smoothing → Conceptual Q
Q3 ~25 min	Neural IR → PyTerrier → 4 Assumptions or HITS

ALWAYS: write formula → substitute → compute → state result. Partial marks!

13. Neural IR — Architectures & Trade-offs

Bi-Encoder (Dense Retrieval):

- Encodes q and d SEPARATELY. Docs pre-indexed offline → fast ANN search
- Use as FIRST STAGE — finds semantic matches (no lexical overlap needed)

Cross-Encoder (Re-ranker):

- Encodes (q,d) JOINTLY — full interaction, best quality
- Cannot pre-compute → slow. Use as RE-RANKER on top-100 only

ColBERT — MaxSim (Late Interaction):

MaxSim(Q,D) = SUM over qi of: MAX over dj of sim(qi, dj) [on cribsheet]

- Doc token embeddings pre-computed offline — best of both worlds

! NaN p-value: re-ranker does NOT change candidate set → Recall@1000

14. PyTerrier Pipelines — Up to 16 marks!

Operators (on cribsheet):

>>	Then — pipe output left to right
%n	Keep top-n results
|	Set Union (combine results)
&	Set Intersection
**	Feature Union for LTR

PRF pipeline:

bm25 % 3 >> rewriter1(index) >> bm25

- rewriter1: sees top-3 docs + index. Uses Bo1 / RM3 (PRF)
- rewriter2 >> bm25: only sees query — uses thesaurus/Word2Vec

15. Dense Retrieval + Passage Aggregation

PROBLEM: BERT 512-token limit, single vector loses long-doc content

FIX: Split into overlapping passages, encode each independently

MaxPassage: score = BEST single passage (one strong section suffices)

SumPassage: score = SUM all passages (evidence spread throughout)

→ Relevant content SPREAD throughout doc? Use SumPassage!

→ One highly relevant section? MaxPassage is fine.